

1. Use LinkedList and Understand Node-Based Storage Concept (Interview Notes)

What is LinkedList?

LinkedList is a **doubly linked list** implementation of the List interface.

Unlike ArrayList, it **does not store elements in a contiguous array**. Instead, each element is stored inside a **node**.

```
LinkedList list = new LinkedList<>();

list.add("Java");
list.add("Spring");
list.add("SQL");
```

How Does LinkedList Store Data?

Each node contains:

- Data
- Reference to previous node
- Reference to next node

```
null ← [Java] ⇌ [Spring] ⇌ [SQL] → null
```

Unlike ArrayList, elements are **linked together** instead of stored next to each other.

Why Use LinkedList?

Use LinkedList when:

- Frequent insertions and deletions are required.
 - Elements are added/removed at the beginning or middle.
 - Random access is not the primary requirement.
-

Important Interview Points

- LinkedList implements both **List** and **Deque** interfaces.
 - Internally implemented as a **doubly linked list**.
 - Each node stores **data + previous pointer + next pointer**.
 - Does **not** require contiguous memory like arrays.
 - Insertion and deletion are efficient once the node is located.
 - Random access is slower because traversal starts from the head or tail.
 - Supports queue and stack operations (addFirst(), addLast(), removeFirst(), etc.).
-

Tricky Interview Questions

Q1. Does LinkedList use an array internally?

Answer: No.

It uses **nodes connected by references**.

Q2. Why is LinkedList slower for get(index)?

Answer:

Because it must traverse nodes one by one.

Time Complexity: **O(n)**.

Q3. Why is insertion faster than ArrayList?

Answer:

No element shifting is required.

Only node references are updated.

Q4. Is LinkedList doubly or singly linked in Java?

Answer: Doubly linked.

Each node has **previous** and **next** references.

Q5. Can LinkedList contain duplicate and null values?

Answer: Yes.

Interview Summary

- **LinkedList** stores data as connected nodes, not as an array.
- Each node contains **data, previous, and next references**.
- Excellent for **frequent insertions and deletions**.
- Poor choice for **random index access** because traversal is required.
- Also implements **Deque**, making it useful for queues and stacks.

Interview One-Liner:

"LinkedList stores elements as doubly linked nodes, making insertions and deletions efficient but random access slower than ArrayList."

2. Compare ArrayList vs LinkedList Using Practical Scenarios (Interview Notes)

Basic Difference

ArrayList	LinkedList
Dynamic Array	Doubly Linked List
Fast random access	Fast insert/delete
Slower middle insertion	Faster middle insertion
Less memory	More memory (extra pointers)

Practical Scenario 1 – Student Records

Need to display student details frequently.

 **Choose ArrayList**

Reason:

- Fast get(index) operation.

- Frequent reading.
-

Practical Scenario 2 – Browser History

Need frequent back/forward navigation.

✓ Choose LinkedList

Reason:

- Frequent insertion and deletion.
 - Easy navigation through linked nodes.
-

Practical Scenario 3 – Music Playlist

Songs are added and removed often.

✓ Choose LinkedList

Practical Scenario 4 – Product Catalog

Mostly searching and displaying products.

✓ Choose ArrayList

Important Interview Points

- ArrayList uses a **dynamic array**; LinkedList uses **doubly linked nodes**.
 - ArrayList provides **$O(1)$** random access; LinkedList provides **$O(n)$** random access.
 - LinkedList performs better for frequent insertions/deletions once the position is reached.
 - ArrayList generally uses **less memory** because it doesn't store node references.
 - ArrayList is the preferred default choice in most applications.
 - Use LinkedList only when insertion/deletion patterns clearly benefit from its structure.
-

Tricky Interview Questions

Q1. Which is faster for `get(index)`?

Answer: ArrayList.

Q2. Which is better for frequent insertions?

Answer: LinkedList.

Q3. Which consumes more memory?

Answer: LinkedList.

Each node stores two extra references.

Q4. Which should you choose by default?

Answer: ArrayList.

Unless your application performs many insertions/deletions.

Q5. Is LinkedList always faster than ArrayList?

Answer: No.

For searching and random access, ArrayList is much faster.

Interview Summary

- **ArrayList** = Fast reading, slower middle modifications.
- **LinkedList** = Fast modifications, slower reading.
- Choose **ArrayList** for search-heavy and read-heavy applications.
- Choose **LinkedList** for applications with frequent insertions and deletions.
- Selection depends on the application's access pattern, not personal preference.

Interview One-Liner:

"Choose ArrayList for fast random access and LinkedList for frequent insertions and deletions."

3. Use HashSet for Uniqueness Without Guaranteed Order (Interview Notes)

What is HashSet?

HashSet is an implementation of the **Set** interface that stores **unique elements**.

It **does not maintain insertion order** and stores elements using a **hash table**.

```
HashSet set = new HashSet<>();

set.add("Java");
set.add("Spring");
set.add("Java");
```

Output

```
Java
Spring
```

Duplicate "Java" is ignored.

Why Use HashSet?

Use HashSet when:

- Duplicate values are not allowed.
 - Fast searching is required.
 - Element order does not matter.
-

Important Interview Points

- Implements the **Set** interface.
- Stores **only unique elements**.
- Uses a **hash table** internally.
- Does **not guarantee insertion order**.
- Allows **one null** element.

- Provides very fast `add()`, `remove()`, and `contains()` operations on average (**$O(1)$**).
 - Uses `hashCode()` and `equals()` to detect duplicate objects.
-

Tricky Interview Questions

Q1. Why doesn't HashSet allow duplicates?

Answer:

It checks objects using `hashCode()` and `equals()` before inserting.

Q2. Does HashSet maintain insertion order?

Answer: No.

Use `LinkedHashSet` if insertion order is required.

Q3. Can HashSet store null?

Answer: Yes.

Only **one null** value.

Q4. Why are `hashCode()` and `equals()` important?

Answer:

They determine whether two objects are considered equal, allowing `HashSet` to prevent duplicates correctly.

Q5. What happens if two objects have the same `hashCode()`?

Answer:

This is called a **hash collision**. `HashSet` then uses `equals()` to determine whether they are actually duplicates.

Q6. Is HashSet sorted?

Answer: No.

Use TreeSet for sorted elements.

Interview Summary

- **HashSet stores only unique elements using a hash table.**
- Does **not guarantee insertion order**.
- Average time complexity for `add()`, `remove()`, and `contains()` is **$O(1)$** .
- Uses **`hashCode()` and `equals()`** to identify duplicates.
- Best choice when **uniqueness and fast lookup** are more important than ordering.

Interview One-Liner:

"HashSet uses hashing to store unique elements with $O(1)$ average lookup, but it does not preserve insertion order."